

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.

Program:

```
import gymnasium as gym
import random

# Create FrozenLake environment
env = gym.make("FrozenLake-v1", is_slippery=False)

# Reset environment
state, info = env.reset()

print("FrozenLake Simulation")

for step in range(20):

    # Choose a random action
    action = random.randint(0, 3)

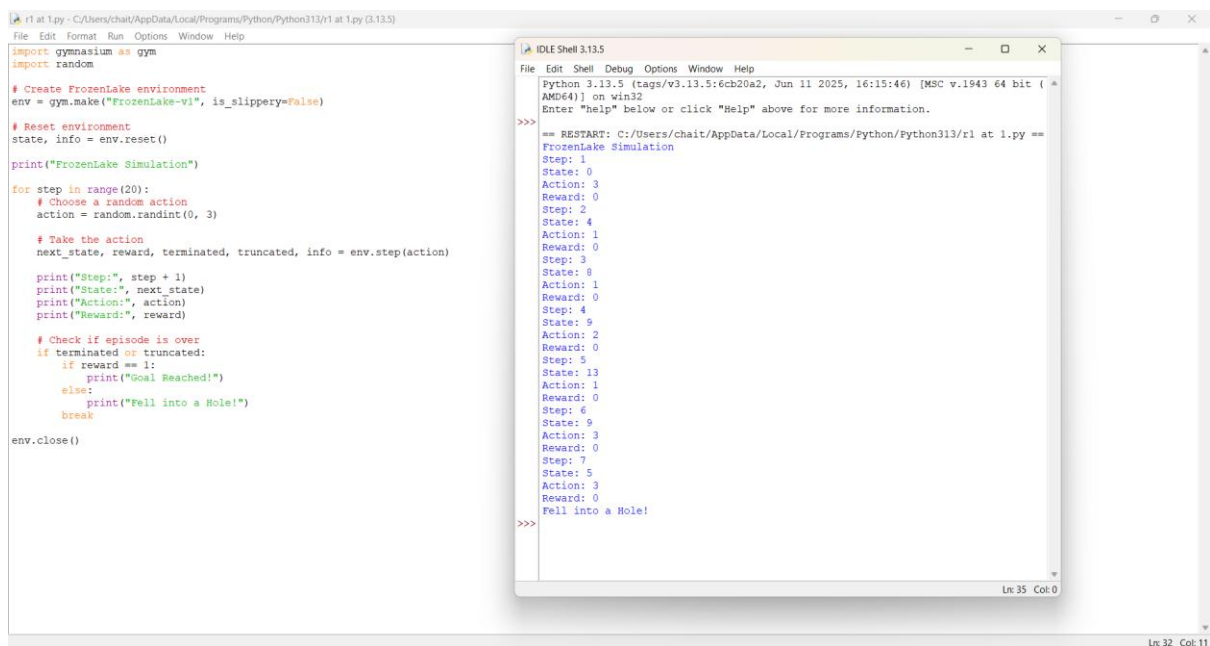
    # Take the action
    next_state, reward, terminated, truncated, info = env.step(action)

    print("Step:", step + 1)
    print("State:", next_state)
    print("Action:", action)
    print("Reward:", reward)

    # Check if episode is over
    if terminated or truncated:
        if reward == 1:
            print("Goal Reached!")
        else:
            print("Fell into a Hole!")
        break

env.close()
```

output:



The image shows a screenshot of a Python script being executed. On the left, a file editor window displays the code for a FrozenLake environment simulation. The code imports 'gymnasium' as 'gym' and 'random'. It creates a 'FrozenLake-v1' environment, resets it, and runs a simulation for 20 steps. In each step, it chooses a random action (0, 1, 2, or 3) and takes it. The environment returns the next state, reward, whether the episode is terminated or truncated, and truncated information. The code prints the step number, state, action, and reward at each step. It also checks if the episode is over (terminated or truncated) and prints a message accordingly. Finally, it closes the environment.

```
import gymnasium as gym
import random

# Create FrozenLake environment
env = gym.make("FrozenLake-v1", is_slippery=False)

# Reset environment
state, info = env.reset()

print("FrozenLake Simulation")

for step in range(20):
    # Choose a random action
    action = random.randint(0, 3)

    # Take the action
    next_state, reward, terminated, truncated, info = env.step(action)

    print("Step:", step + 1)
    print("State:", next_state)
    print("Action:", action)
    print("Reward:", reward)

    # Check if episode is over
    if terminated or truncated:
        if reward == 1:
            print("Goal Reached!")
        else:
            print("Fell into a Hole!")
        break

env.close()
```

On the right, the IDLE Shell window shows the output of the script. It starts with a restart message, followed by the simulation output for 20 steps. The output shows the state, action, and reward for each step. The simulation ends with the message "Fell into a Hole!".

```
>>> == RESTART: C:/Users/cha1t/AppData/Local/Programs/Python/Python313/r1 at 1.py ==
FrozenLake Simulation
Step: 1
State: 0
Action: 3
Reward: 0
Step: 2
State: 4
Action: 1
Reward: 0
Step: 3
State: 8
Action: 1
Reward: 0
Step: 4
State: 9
Action: 2
Reward: 0
Step: 5
State: 13
Action: 1
Reward: 0
Step: 6
State: 9
Action: 3
Reward: 0
Step: 7
State: 5
Action: 3
Reward: 0
Fell into a Hole!
>>>
```

2. Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

Program:

```
import numpy as np

np.random.seed(42)

true_rewards = [1.2, 1.5, 1.8, 1.0, 2.0]

trials = 500

def epsilon_greedy(epsilon):

    k = len(true_rewards)

    Q = np.zeros(k)

    N = np.zeros(k)

    total_reward = 0

    for i in range(trials):

        if np.random.rand() < epsilon:

            action = np.random.randint(k)
```

```

else:

    action = np.argmax(Q)

    reward = np.random.normal(true_rewards[action],0.2)

    N[action]+=1

    Q[action]+= (reward-Q[action])/N[action]

    total_reward += reward

return total_reward

for e in [0.1,0.3,0.5]:

    reward = epsilon_greedy(e)

    print("Epsilon =",e," Total Reward =",round(reward,2))

```

output:

```

r1 at 1.py - C:/Users/cha1/AppData/Local/Programs/Python/Python313/r1 at 1.py (3.13.5)
File Edit Format Run Options Window Help
import numpy as np
np.random.seed(42)
true_rewards = [1.2, 1.5, 1.8, 1.0, 2.0]
trials = 500
def epsilon_greedy(epsilon):
    k = len(true_rewards)
    Q = np.zeros(k)
    N = np.zeros(k)
    total_reward = 0
    for i in range(trials):
        if np.random.rand() < epsilon:
            action = np.random.randint(k)
        else:
            action = np.argmax(Q)
        reward = np.random.normal(true_rewards[action],0.2)
        N[action]+=1
        Q[action]+= (reward-Q[action])/N[action]
        total_reward += reward
    return total_reward
for e in [0.1,0.3,0.5]:
    reward = epsilon_greedy(e)
    print("Epsilon =",e," Total Reward =",round(reward,2))

```

```

IDLE Shell 3.13.5
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/cha1/AppData/Local/Programs/Python/Python313/r1 at 1.py ==
Epsilon = 0.1 Total Reward = 966.85
Epsilon = 0.3 Total Reward = 929.62
Epsilon = 0.5 Total Reward = 876.77
>>>

```

3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

Program:

```

# Markov Decision Process (MDP) for an Autonomous Robot

states = ["Start", "Road1", "Road2", "Charging", "Goal"]

actions = ["Forward", "Left", "Right", "Recharge"]

# Transition probabilities

```

```
transition = {  
    ("Start", "Forward"): {"Road1": 0.9, "Start": 0.1},  
    ("Road1", "Forward"): {"Road2": 0.8, "Charging": 0.2},  
    ("Road2", "Forward"): {"Goal": 0.9, "Road2": 0.1},  
    ("Charging", "Recharge"): {"Road2": 1.0}  
}
```

```
# Rewards
```

```
rewards = {  
    "Goal": 100,  
    "Road1": -2,  
    "Road2": -2,  
    "Charging": -5,  
    "Start": 0  
}
```

```
battery = 100    # Initial battery level
```

```
energy_per_move = 5
```

```
current_state = "Start"
```

```
total_reward = 0
```

```
print("Robot Navigation using MDP\n")
```

```
while current_state != "Goal" and battery > 0:
```

```
    if current_state == "Charging":
```

```
        action = "Recharge"
```

```
    else:
```

```
        action = "Forward"
```

```
    print("Current State:", current_state)
```

```
    print("Action:", action)
```

```
    next_states = transition.get((current_state, action), {})
```

```
    if next_states:
```

```

        # Choose the state with the highest probability
        next_state = max(next_states, key=next_states.get)

    else:

        next_state = current_state

    battery -= energy_per_move

    total_reward += rewards.get(next_state, -2)

    print("Next State:", next_state)

    print("Battery Remaining:", battery)

    print("Reward:", rewards.get(next_state, -2))

    print("-" * 35)

    current_state = next_state

print("\nFinal State:", current_state)

print("Total Reward:", total_reward)

```

output:

The screenshot shows a Python script in a file editor on the left and its execution output in a terminal window on the right. The script implements a robot navigation algorithm using a Markov Decision Process (MDP). It starts with a battery level of 100 and a goal state. The robot moves through states (Start, Road1, Road2, Goal) and receives rewards. The output shows the sequence of states, actions, and rewards, ending with a final state of Goal and a total reward of 96.

```

# Rewards
rewards = {
    "Goal": 100,
    "Road1": -2,
    "Road2": -2,
    "Charging": -5,
    "Start": 0
}

battery = 100 # Initial battery level
energy_per_move = 5
current_state = "Start"
total_reward = 0

print("Robot Navigation using MDP\n")
while current_state != "Goal" and battery > 0:
    if current_state == "Charging":
        action = "Recharge"
    else:
        action = "Forward"
    print("Current State:", current_state)
    print("Action:", action)
    next_states = transition.get((current_state, action), {})
    if next_states:
        # Choose the state with the highest probability
        next_state = max(next_states, key=next_states.get)
    else:
        next_state = current_state
    battery -= energy_per_move
    total_reward += rewards.get(next_state, -2)
    print("Next State:", next_state)
    print("Battery Remaining:", battery)
    print("Reward:", rewards.get(next_state, -2))
    print("-" * 35)
    current_state = next_state

print("\nFinal State:", current_state)
print("Total Reward:", total_reward)

```

```

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> == RESTART: C:/Users/chaht/AppData/Local/Programs/Python/Python313/r1 at 1.py ==
Epsilon = 0.1 Total Reward = 966.85
Epsilon = 0.3 Total Reward = 929.62
Epsilon = 0.5 Total Reward = 876.77

>>> == RESTART: C:/Users/chaht/AppData/Local/Programs/Python/Python313/r1 at 1.py ==
Robot Navigation using MDP

Current State: Start
Action: Forward
Next State: Road1
Battery Remaining: 95
Reward: -2
-----
Current State: Road1
Action: Forward
Next State: Road2
Battery Remaining: 90
Reward: -2
-----
Current State: Road2
Action: Forward
Next State: Goal
Battery Remaining: 85
Reward: 100
-----
Final State: Goal
Total Reward: 96
>>>

```

4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

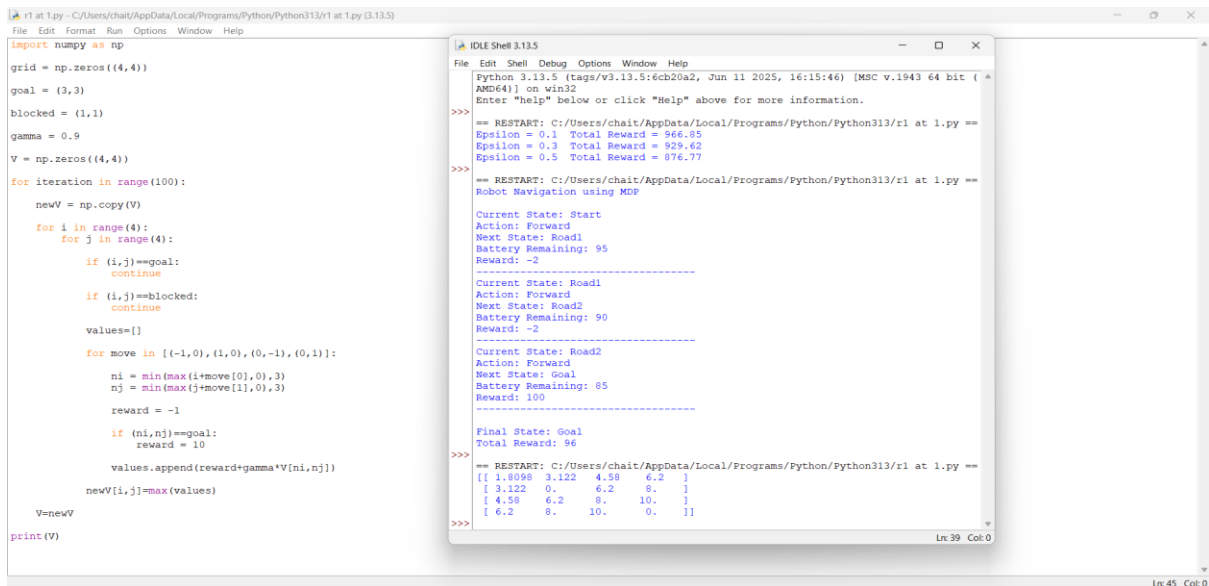
Program:

```

import numpy as np
grid = np.zeros((4,4))
goal = (3,3)
blocked = (1,1)
gamma = 0.
V = np.zeros((4,4))
for iteration in range(100):
    newV = np.copy(V)
    for i in range(4):
        for j in range(4):
            if (i,j)==goal:
                continue
            if (i,j)==blocked:
                continue

            values=[]
            for move in [(-1,0),(1,0),(0,-1),(0,1)]:
                ni = min(max(i+move[0],0),3)
                nj = min(max(j+move[1],0),3)
                reward = -1
                if (ni,nj)==goal:
                    reward = 10
                values.append(reward+gamma*V[ni,nj])
            newV[i,j]=max(values)
    V=newV
print(V)
output:

```



5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

Program:

```
import random
```

```
# Initial settings
```

```
battery = 30      # Battery capacity in minutes
```

```
deliveries = 0
```

```
total_reward = 0
```

```
print("Warehouse Robot Reinforcement Learning\n")
```

```
# Continue until battery is exhausted
```

```
while battery > 0:
```

```
    # Randomly choose an action (policy)
```

```
    action = random.choice(["Deliver", "Recharge", "Wait"])
```

```
    if action == "Deliver":
```

```
        if battery >= 5:
```

```
            battery -= 5
```

```
        deliveries += 1

        reward = 20

        print("Action: Deliver Package")

    else:

        reward = -50

        print("Battery too low for delivery!")

elif action == "Recharge":

    reward = -5

    battery = min(30, battery + 10)

    print("Action: Recharge Battery")

else: # Wait

    battery -= 1

    reward = -2

    print("Action: Wait")

total_reward += reward

print("Battery:", battery, "minutes")

print("Reward:", reward)

print("Deliveries:", deliveries)

print("-" * 35)

# Stop if battery is empty

if battery <= 0:

    print("Battery exhausted!")

    break

print("\nSimulation Completed")

print("Total Deliveries:", deliveries)

print("Total Reward:", total_reward)
```


output:

```
r1 at 1.py - C:/Users/chat/AppData/Local/Programs/Python/Python313/r1 at 1.py (3.13.5)
File Edit Format Run Options Window Help
battery = 30          # Battery capacity in minutes
deliveries = 0
total_reward = 0

print("Warehouse Robot Reinforcement Learning\n")

# Continue until battery is exhausted
while battery > 0:

    # Randomly choose an action (policy)
    action = random.choice(["Deliver", "Recharge", "Wait"])

    if action == "Deliver":
        if battery >= 5:
            battery -= 5
            deliveries += 1
            reward = 20
            print("Action: Deliver Package")
        else:
            reward = -50
            print("Battery too low for delivery!")

    elif action == "Recharge":
        reward = -5
        battery = min(30, battery + 10)
        print("Action: Recharge Battery")

    else: # Wait
        battery = 1
        reward = -2
        print("Action: Wait")

    total_reward += reward

    print("Battery:", battery, "minutes")
    print("Reward:", reward)
    print("Deliveries:", deliveries)
    print("-" * 35)

    # Stop if battery is empty
    if battery <= 0:
        print("Battery exhausted!")
        break

print("\nSimulation Completed")
print("Total Deliveries:", deliveries)
print("Total Reward:", total_reward)
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Battery: 18 minutes
Reward: 20
Deliveries: 19
-----
Action: Deliver Package
Battery: 13 minutes
Reward: 20
Deliveries: 20
-----
Action: Deliver Package
Battery: 8 minutes
Reward: 20
Deliveries: 21
-----
Action: Wait
Battery: 7 minutes
Reward: -2
Deliveries: 21
-----
Action: Deliver Package
Battery: 2 minutes
Reward: 20
Deliveries: 22
-----
Action: Wait
Battery: 1 minutes
Reward: -2
Deliveries: 22
-----
Action: Wait
Battery: 0 minutes
Reward: -2
Deliveries: 22
-----
Battery exhausted!
Simulation Completed
Total Deliveries: 22
Total Reward: 284
>>>
```

Ln 323 Col 0

Ln 51 Col 0